os384 Root Security — Design Document

RC3 Draft

os384 design notes

2026-04-17

Table of Contents

os384 Root Security — Design Document

Status: draft for internal review (RC3 cycle) **Audience:** mixed — technically proficient readers who may or may not be security specialists, and/or may or may not be familiar with os384. Appendix A describes os384 in enough detail for an unfamiliar reader to follow. Appendix B explains security concepts referenced throughout. Footnote pointers like [A], [B§SRI] appear inline where useful.

---

0. Summary

os384 is a browser-native, sovereign-computing platform built on a small set of cryptographic primitives (SB384 identities, P-384 ECDSA, content-addressed shards, owner-keyed channels, public pages). It has always held "trust as little as possible" as a design ethos, and most of the building blocks needed to realise that ethos end-to-end are already present in the codebase. What it does *not* yet have is a **coherent articulated strategy** for root security — the chain of trust from "a user opens a browser" or "an operator clones the service repo" through to "a piece of os384 code is executing the user's intent." Individual pieces of the stack make varying, sometimes implicit, trust assumptions; some of those assumptions work against the platform's own ethos.

This document takes a step back from any particular implementation ticket. It:

1. Summarises os384 (§2, Appendix A).
2. Enumerates the classical security problems that compute platforms have learned, painfully, to take seriously (§3).
3. Identifies where os384 currently stands relative to each (§4–5).
4. Derives design principles from the platform's ethos plus the gap analysis (§6).
5. Names the decision space — distinguishing genuine open questions from preferences that follow directly from the existing stack (§7).

6. Offers one concrete path ("Proposal A") that instantiates the principles and could land in RC3 (§8), plus alternatives considered (§9), success criteria (§10), and what is explicitly deferred (§11).

This is a design document, not a work plan. Where Proposal A references concrete artifacts, those references are illustrative of the design, not commitments to an implementation schedule.

---

1. Purpose and audience

Two overlapping audiences are expected:

- A **technically proficient reader** — e.g. a CS-trained engineer or collaborator — who wants to understand where os384's security story is heading, and whether the direction is reasonable. This reader may not be a security specialist and may be new to os384. Appendix A and Appendix B are written for this reader.
- A **security specialist** who wants to evaluate whether os384 is thinking coherently about root trust. This reader may or may not know os384 in detail, but will care about how the design addresses well-understood threat categories, which choices are load-bearing, and where the honest caveats are. §3 (problem space), §4–5 (gap analysis), §11 (deferred), and Appendix B should be the main points of engagement.

The document is *not* a comprehensive threat model, audit, or formal specification. It aims to make the design space legible, expose the reasoning behind any concrete proposal, and provide a structure for further review.

---

2. os384 at a glance

A minimum sketch to make the rest of the document readable; Appendix A has the longer version.

- **os384** is a browser-native sovereign-computing platform. Its goal is to let users own their data and their compute, with trust placed in as few third parties as possible.
- **SB384** is the root identity object: a P-384 ECDSA keypair, addressable by hashes of its public key.
- **SBCrypto** provides sign/verify (ECDSA P-384 / SHA-384), plus symmetric primitives.
- **Shards** are padded, content-addressed, AES-256-GCM encrypted blobs. Their content-addressing makes them self-verifying against bit-level corruption or substitution.
- **Channels** are owner-keyed, end-to-end-encrypted, addressable-worker-backed endpoints for interactive, stateful use.
- **Pages** are public-facing content served at /api/v2/page/<pageKey>, where pageKey = base62(SHA-384(ownerPubKey)). Pages are how os384 publishes things like its own library code, service-worker code, and application-level content to the open web.

- **Runtimes**: os384 code runs in browsers (lib384), in Cloudflare Workers (channel / storage / mirror / admin services), in a local CLI, and in a Docker compose stack for self-hosters.
- **Hosting reality today**: the canonical public entry point is 384.dev, served via Cloudflare Pages/Workers.

For this document, the load-bearing facts are: every os384 identity has a P-384 keypair; every owner's public key has a canonical short hash (pageKey) that appears in URLs; and the platform already has sign/verify primitives wired, even where they are not currently used.

---

3. The problem space: classical security lessons for compute platforms

A century of (sometimes expensive) industry experience has clarified which trust problems recur. The categories below are not exhaustive; they are the ones that bear on a platform like os384.

3.1 Code distribution integrity

- **CDN / infrastructure injection.** A hosting provider, caching proxy, or ISP can modify code in transit or at rest. Cloudflare's Rocket Loader and similar "helpful" features inject JavaScript by default; the provider could, in principle, inject anything.
- **Supply-chain attacks.** Compromised dependencies at build time (npm, PyPI, OS package managers) substitute malicious code into otherwise legitimate artifacts. Recent notable cases include event-stream, xz-utils, and SolarWinds. A build can pass tests, sign correctly, and still ship an attacker's payload [B§SupplyChain].
- **Update-channel compromise.** Even a correctly signed update system can be abused if the signing authority itself is compromised or coerced (e.g. CrowdStrike incident by operational failure; targeted attacks on vendor signing).
- **Reproducible builds.** When the same source can produce different binaries, there is deniable space between "source looked fine" and "binary is bad." Reproducibility is a precondition for meaningful community audit [B§ReproducibleBuilds].

3.2 Transport trust

- **DNS hijack.** If an attacker controls DNS for example.com, they control what https://example.com/… resolves to. DNSSEC and registry locks reduce but do not eliminate risk.
- **TLS / CA trust.** TLS guarantees the channel to *some* party holding a valid certificate. CA compromise (DigiNotar), mis-issuance, or coerced issuance can give the wrong party a valid cert. Certificate Transparency logs [B§CT] mitigate this partially.
- **HTTP-header reliance.** Security headers (CSP, SRI, etc.) delivered only in HTTP response headers are under the delivery infrastructure's control; they can be stripped or weakened.

3.3 Bootstrap trust (the "loader-for-the-loader" problem)

- **Trust on first use (TOFU).** The first time a user interacts with a system, they have no existing anchor; they are forced to trust DNS + TLS + the server implicitly. Subsequent pins can improve things, but the first fetch is exposed.
- **Bootstrap circularity.** Code that verifies other code must itself be verified by something. This regress has to bottom out somewhere — in hardware, in OS code-signing, in a browser's code, in a user's filesystem, in a published hash.
- **"But I'm loading the verifier over the same channel."** Using a system's own verification code to verify that system's own delivery is circular unless the verifier is fetched through an independent, integrity-enforcing path.

3.4 Identity and key management

- **Key compromise.** Any private key can leak. The question is not "if" but "what happens when."
- **Rotation.** A system with no rotation story is, after a long enough time horizon, a system with a stale compromised key.
- **Revocation.** Declaring that "key X is no longer trusted" is a communication problem, and a surprisingly unsolved one at platform scale.
- **Per-purpose separation.** Conflating signing, encryption, and authentication keys increases blast radius of compromise.
- **Multi-party / threshold.** When a single signing key is catastrophic to lose, threshold or multi-sig schemes reduce single-point risk.

3.5 Bootloader and chain-of-trust analogues

Classical OSes and devices implement chains: hardware root → firmware → bootloader → kernel → userland → app, each step verifying the next before transferring control. Each link is only as strong as the next, and the root has to be physically or cryptographically anchored to something outside the control of any online attacker (TPM, Apple Secure Enclave, Android Verified Boot, UEFI Secure Boot).

A web-native platform has no equivalent hardware root; the best analogues are browser-enforced primitives (SRI, CSP), user-controlled artifacts (saved HTML, browser extensions, installed native apps), and publicly published hashes that a community can audit.

3.6 Runtime integrity and isolation

- **Sandboxing.** Can malicious code, if it does run, be constrained? Browsers, containers, and operating systems offer varying degrees of isolation.
- **Process boundaries and IPC.** Which component can lie to which?
- **Persistent compromise.** Service workers, cached pages, local storage — once a malicious payload is cached, it may persist past the incident that delivered it.

3.7 Replay and downgrade

- **Serving a known-bad previous version.** Even if every version is validly signed, an attacker who serves v1.0 instead of v1.2 after v1.1 was found vulnerable still wins, unless monotonic version enforcement is in place.

- **Protocol downgrade.** Advertising an older, weaker negotiation option and coaxing the client to accept it.

3.8 Auditability

- **Transparency logs.** CT, sigstore Rekor, and similar systems publish append-only records so third parties can detect surprise issuance or publication [B§CT].
- **Reproducibility plus publication.** If the source, build process, and output are all public and the build is reproducible, independent parties can verify that the published binary matches the source.
- **Signed-but-untrusted.** A valid signature from an unknown signer is worth little; the question "who is authorised to sign this?" is often harder than the cryptography.

3.9 Social and operational

- **Insider risk** at hosting, CA, or vendor level.
- **Coercion** of signing-key holders by legal process or force.
- **Single-point community** where one maintainer takeover compromises a dependency used by thousands (xz-utils).
- **Adoption gap** between the paranoid option (native app, extension, air-gap) and the default option (web URL): if defaults are weak, most users are exposed; if defaults are too strict, the platform is unusable.

---

4. os384's guiding philosophy

"Trust as little as possible." In practical terms this manifests as:

- Every user is their own root of identity (their SB384 keypair), not a platform-issued account.
- Storage is content-addressed and owner-encrypted: the storage operator is not trusted for confidentiality or integrity of shard contents.
- Identity is derived from keys, not from server-issued tokens: the server is not trusted to assert who someone is.
- Clients should be able to verify what the server claims, not take it on faith.

This philosophy is strong where it is wired end-to-end (shards: good), mixed where primitives exist but are not wired (signing: partial), and weak where trust is left implicit (entry HTML on 384.dev, pages, runtime fetches in the Docker bootstrap).

---

5. os384 primitives — what helps, what's present, what's missing

What is already in the codebase that the root-security story can build on:

- **P-384 asymmetric crypto**, exposed via SBCrypto.sign and SBCrypto.verify using ECDSA + SHA-384. Sufficient for signing arbitrary byte blobs.

- **SB384 root identity object.** Every user and every service has one; SB384.signKey surfaces a CryptoKey ready for sign/verify.
- **The pageKey = base62(SHA-384(pubkey)) convention.** Already embedded in every page URL. This is a free-on-the-floor identity binding: if a client knows the pageKey it expects, and recomputes SHA-384(pubkey) on whatever public key the server hands back, the server cannot substitute the owner without being caught.
- **Shards.** Already content-addressed, already self-verifying. A signed or verified reference to a shard is a strong primitive for "get this exact content."
- **Service worker**, installable and persistent once registered, offering a chokepoint for future intercept-and-verify policies — but requiring a trusted first install.
- **CLI publish tooling** (384 publish) already handles the page-upload path, so it is the natural place to insert signing.

What is missing today:

- **No signature verification anywhere on the page-fetch path.** A client fetching a page gets raw bytes with a server-computed hash for caching purposes; nothing binds those bytes to the owner's signature. A compromised channel server can serve arbitrary content under any pageKey.
- **No hardened bootstrap for browser entry.** 384.dev serves HTML through Cloudflare without an articulated integrity story — CDN injection is possible in principle, and the platform has no documented guidance to users on how to harden their entry point.
- **Runtime fetches in the Docker self-host path.** services/docker/channels/start.sh, storage/start.sh, and loader/start.sh download worker code from c3.384.dev and 384.dev at container boot and exec it without verification. Self-hosting does not yet imply self-sovereignty.
- **No canonical "this is the expected owner-key for X" registry.** Consumers have no standard artifact to pin against.
- **No published release-signing discipline.** Git tags are unsigned; canonical hashes for built artifacts (lib384 bundle, service worker, Docker tarballs) are not published for community audit.
- **No stated trust tiers** for users who want to operate at different paranoia levels (default web, saved-to-disk, self-hosted, extension, native).
- **No rotation, revocation, or transparency-log story.**

This is not a criticism of prior work — the primitives needed to fix most of these are already in the codebase, and doing it prematurely would have been the wrong investment. RC3 is a reasonable point to articulate the strategy and wire the primitives end-to-end.

---

6. Gap analysis — mapping §3 onto os384

Classical problem (§3)
3.1 CDN / infrastructure injection

3.1 Supply-chain at build time
3.1 Update-channel compromise
3.1 Reproducible builds
3.2 DNS hijack
3.2 CA compromise
3.2 Header reliance
3.3 TOFU at entry
3.3 Bootstrap circularity
3.4 Key compromise
3.4 Rotation
3.4 Revocation
3.4 Per-purpose separation
3.4 Multi-party signing
3.5 Chain of trust
3.6 Sandboxing
3.6 Persistent compromise
3.7 Replay / downgrade
3.8 Transparency logs
3.8 Reproducibility + publication
3.8 Signed-but-untrusted
3.9 Insider / coercion / single-point
3.9 Adoption gap

The striking pattern: **most gaps share a common root** — the lack of a small, explicit, auditable trust anchor that lets clients bind what they fetch to an owner's identity. Fixing that common root closes a disproportionate share of the gaps in one move, and makes the remaining gaps (rotation, transparency log, reproducibility) well-defined follow-on problems rather than tangled ones.

---

7. Design principles for os384 root security

These are derived from §4 (philosophy) and §6 (gap analysis). They are normative — any concrete proposal should hold these, or give a reason for not holding them.

1. **One explicit trust anchor per user deployment.** A user, at a given paranoia tier, should be able to point to exactly the file(s) / hash(es) / keys that are ambient trust for their session. Anything not in that set should be cryptographically bound to it.
2. **No ambient trust that isn't visible.** If the design depends on trusting 384.dev, say so; if it does not, prove so by mechanism. No "it's probably fine" in the chain.

3. **The verifier does not depend on the thing it verifies.** Breaks bootstrap circularity. The path that gets the verifier into the user's execution environment must be independent from the path that delivers verified content.
4. **Failure is loud.** Verification mismatches throw; they do not silently fall back. Diagnostics are specific enough to let an operator identify the failure mode in one look.
5. **One verification story, many consumption paths.** The same envelope format, the same verifier logic, and the same trust-anchor discipline serve browsers, containers, CLI operators, and (eventually) native apps. Divergent schemes multiply surface area.
6. **Use existing primitives; don't invent new ones unless required.** P-384 / SHA-384 / AES-256-GCM / content-addressed shards already exist. Avoid introducing new key types, new hash functions, new custom formats unless a concrete gap demands it.
7. **Tier the trust story; don't force one answer.** Users have varying threat models. The default (web URL) should be reasonably hardened; progressively stronger options (save-to-disk, self-host, extension, native) should exist, use the same mechanisms, and require progressively more operator effort.
8. **Breaking changes are acceptable in beta** when they meaningfully close a gap — provided the migration story is documented and the post-change state is coherent.
9. **Defer honestly.** Naming a gap explicitly deferred (rotation, replay enforcement, transparency log) is better than leaving it ambiguous. Deferred items get a rationale and an intended horizon.
10. **Publication is part of the design.** Canonical hashes, signed release tags, and documentation are not afterthoughts — they are the audit surface that lets the community verify the platform.

---

8. Proposal A: one concrete instantiation

This section presents one design that holds Principles 1–10 and closes the largest share of §6's gaps at once. It is offered as a baseline for discussion, not the only possible answer. Alternatives considered are in §9.

8.1 Shape of the proposal

Five coupled moves:

1. **Sign every page.** Introduce a signed outer envelope for content served via pages. Signatures are generated by the page owner with their SB384 key, verified by the client at fetch time. The existing pageKey = SHA-384(pubkey) convention becomes the enforced bootstrap binding: a client that knows its expected pageKey cannot be tricked into trusting a substituted owner.
2. **Separate verifier artifact ("libLoader").** Provide a small, self-contained verifier — not dependent on lib384, parseable in plain JavaScript + Web Crypto, usable by a browser and by Node in a container. This breaks bootstrap circularity (Principle 3).

3. **Distribute the verifier with browser-enforced integrity.** Pin the verifier via Subresource Integrity [B§SRI] on the entry HTML. Couple with strict Content Security Policy [B§CSP] to block injected scripts (addresses §3.1 CDN injection).
4. **Harden the entry HTML and offer trust tiers** (Principle 7): Tier 0 hosted (384.dev with CSP + SRI), Tier 1 saved-to-disk static file, Tier 2 self-hosted loader, Tier 3 browser extension (future), Tier 4 native app.
5. **Make Docker self-host meaningful.** Use the same signed-envelope + verifier scheme for worker code fetched at container boot; pin expected owner hashes in a committed pins.json; offer an "offline tarball" that ships verified artifacts rather than recipes.

And one cross-cutting discipline:

6. **Signed release tags and published canonical hashes.** Every RC3 artifact has a signed git tag; release notes include the SHA-384 of the entry HTML, the verifier, the lib384 bundle, the service-worker, and the Docker tarball. This is the audit surface.

8.2 Why these five, together

Each move on its own is insufficient:

- Signed pages without a separate verifier re-introduce bootstrap circularity.
- A separate verifier without SRI gives the CDN control of what "the verifier" actually is.
- A hardened entry without signed content still lets a compromised server substitute libraries under valid SRI-loaded framing.
- Trust tiers without a common mechanism become divergent code and divergent bugs.
- Docker self-host without verified bootstrap is a worse trust story than the default, not a better one.

They are designed to compose: one envelope format, one verifier, one trust-anchor discipline, multiple tiers.

8.3 The envelope (tentative)

A fixed-offset binary shell whose goal is to be parseable by a ~60-line verifier using only Web Crypto, with the body opaque to the verifier:

```
offset  size    field            notes
-----------------------------------------------------------
0       4       magic            ASCII 'S3V1'
4       4       envelopeVersion  LE uint32, = 1
8       4       contentVersion   LE uint32, author-chosen monotonic
12      2       pubkeyLen        LE uint16
14      2       bodyTypeLen      LE uint16
16      4       bodyLen          LE uint32
20      pkLen   pubkey           JWK JSON, UTF-8
20+pk   btLen   bodyType         MIME string, UTF-8
...     bLen    body             opaque content bytes
...     96      signature        ECDSA P-384/SHA-384 over bytes[0:sigStart]
```

Rationale:

- **Fixed-offset** so the verifier can extract fields with only DataView — no dependency on assemblePayload or other lib384 format helpers (Principle 3).
- **Opaque body** so the envelope serves JS, JSON, WASM, images, or nested sb384 payloads without the verifier needing to understand the content.
- **Deliberately not executable as raw JS.** The only way to obtain executable content is through the verifier. This prevents the footgun where consumers forget to verify.
- **contentVersion present but not enforced by default.** Monotonic pinning is available to consumers who want replay protection; default consumers can observe the version without being locked in.
- **Preferenced choices carried from §5 primitives**: P-384 / SHA-384, JWK for pubkey encoding, 96-byte raw ECDSA signature.

Things that are *tentative* in this shape: field widths (uint16 vs uint32), whether to include an explicit bodyHash for sub-body streaming verification, whether the signed region should include the magic/version bytes (currently yes), whether pubkeyJWK or raw EC point encoding is better for audit legibility. All genuine open questions.

8.4 The verifier (tentative)

A single-source module with thin browser and Node wrappers, roughly 60 lines of logic:

1. Check magic, version, length consistency.
2. Compute SHA-384(pubkey) and compare (byte-equal) to a caller-supplied expectedOwnerHashHex — the **bootstrap binding**. This is the step that defeats owner substitution by a compromised server.
3. Import the pubkey as an ECDSA verification key.
4. Verify the signature over the signed region.
5. Return {body, bodyType, contentVersion, publicKeyJWK} or throw.

Tentative naming: libLoader (to avoid collision with the kernel/VMM-level "loader" concept in os384).

8.5 Distribution: SRI-pinned verifier + strict CSP entry HTML

The entry HTML (web384.html, ≤ 50 source lines) declares a strict CSP [B§CSP] and contains exactly one <script src> — the verifier, integrity-pinned [B§SRI]. Canonical shape:

```
<meta http-equiv="Content-Security-Policy"
    content="default-src 'none';
            script-src 'sha384-<verifierHash>' 'self';
            connect-src https://*.384.dev;
            worker-src 'self';
            style-src 'self';
            base-uri 'none'; form-action 'none'; frame-ancestors 'none';">
<script src="/libLoader-v01.js"
```

```
      integrity="sha384-<verifierHash>"
      crossorigin="anonymous"></script>
<script>
  // tiny bootstrap: hardcoded expectedOwnerHashHex for current lib384
  window.__libLoader__.importSignedLibrary(
    '/api/v2/page/<lib384PageKey>',
    '<lib384OwnerHashHex>'
  ).then(lib => { /* ... */ });
</script>
```

The two load-bearing hashes — verifierHash and lib384OwnerHashHex — are visible in the HTML source, auditable by View-Source, and pinned per release. Cloudflare features that inject scripts (Rocket Loader, Auto-Minify on this path) are disabled at zone level and/or refused by CSP.

8.6 Trust tiers

Tier
0
1
2
3
4

All tiers consume the same signed-envelope format and verifier — Principle 5. Moving tiers is a user decision, not a code change.

8.7 Docker self-host / "secure enclave"

The current services/docker/ stack fetches worker code at container boot without verification. Proposal A treats those fetches the same way as a browser page load:

- Fetched bytes are sb384signedV1 envelopes.
- A committed services/docker/pins.json lists expected owner-hash-hex values per artifact.
- start.sh scripts verify before exec wrangler dev.
- A new make offline-tarball target resolves fetches at build time and packs the already-verified envelopes, producing a self-contained bootable artifact for air-gapped deployment.
- Dockerfile pins the base image by digest.

"Secure enclave" overstates it (no hardware attestation), but the property delivered is meaningful: an advanced user who clones services at a signed git tag and runs docker compose up has a trust story that depends on the tag, the pins, the base-image digest, and their local machine — nothing else. A compromised 384.dev cannot affect their running stack.

8.8 Release discipline

- Signed git tags (cosign or GPG — open decision) on lib384, services, os384, loader-v02.
- RC3 release notes publish:
  - SHA-384 of web384.html
  - SHA-384 of libLoader-v01.js
  - SHA-384 of the lib384 bundle + its owner hash
  - SHA-384 of the service-worker
  - SHA-384 of os384docker.<date>.offline.tar.gz
- Any subsequent fix to a pinned artifact bumps the version number; pinned bytes never change after publication.

This is not a full transparency log [B§CT], but it is the precondition for one and is immediately useful to community auditors.

8.9 What Proposal A explicitly does *not* attempt

(Principle 9.)

- **Strict replay enforcement.** contentVersion is carried and available; default consumers do not enforce monotonicity. A follow-on design can layer this on without envelope changes.
- **Key rotation / revocation.** A compromised signing key requires out-of-band coordination in this design. A rotation protocol is a separate design problem and a post-RC3 priority.
- **Multi-party / threshold signing.** Single-key signing today.
- **Transparency log infrastructure.** Release-note-published hashes + signed git tags are a "zeroth approximation" to a CT-like log [B§CT]. A real log (append-only, externally witnessed) is a post-RC3 effort.
- **Hardware-TEE / remote attestation.** Not in scope.
- **Supply-chain audit (lockfile vetting, reproducible builds)**. Worth a separate design exercise; noted as a gap.

---

9. Alternatives considered

9.1 Use the existing `sb384payloadV3` format for signed pages

Appealing for uniformity, but the verifier would need an assemblePayload parser — pulling lib384 format logic into the bootstrap, re-introducing circularity (Principle 3). Rejected in Proposal A. A follow-on can layer assemblePayload *inside* the body of a signed envelope if richer internal structure is desired, without changing the outer shell.

9.2 Dual-loadable envelope (plaintext JS with trailing signature comment)

Technically feasible (source-map-comment style) and would let the same bytes be loaded via <script src> or via the verifier. Rejected because it creates a footgun: consumers who *forget* to verify get silently unverified code, which is exactly the failure mode we are defending against.

9.3 Server-side verification only

Trivially bypassable by a compromised server that lies about having verified. Server-side verify has value only as defense-in-depth at publish time (setPage) to catch malformed/broken publishes early; it cannot substitute for client-side verify. Proposal A treats server-side verify as an optional polish item.

9.4 Signed manifests pointing at shards

A signed manifest {shardRef, bodyHash, version, signature, pubkey} reuses shard infrastructure's existing content-addressing. Attractive for versioning and rotation (sign a new manifest pointing at a new shard). Rejected as the *baseline* because it adds a round-trip and introduces a plaintext-shard pattern that is orthogonal to the main problem. A worthwhile follow-on design for the library-update flow specifically.

9.5 Browser extension as the default

Would give the strongest integrity story (extension bundles are vendor-signed). Rejected as the default because of the adoption barrier (Principle 7 / §3.9). Retained as Tier 3.

9.6 Rely on service-worker for verification

A registered service-worker can intercept all future fetches. Useful as defense-in-depth after install, but does not help the first-install problem (the service-worker itself needs to be verifiably delivered).

---

10. Success criteria

RC3 succeeds on root security if, after the release:

1. Every page served by os384 channel infrastructure is in a signed envelope format; the legacy unverified format is not accepted by RC3 clients.
2. No browser or container path that executes os384 code does so without prior signature verification.
3. The verifier is a reviewable standalone artifact, pinnable via SRI, not dependent on lib384.
4. The 384.dev entry HTML declares strict CSP, references the verifier via SRI, and fits in a page of source.
5. A Tier 1 (save-to-disk) static entry is published; its canonical hash appears in release notes.

6. Docker self-host verifies fetched worker code; an offline-tarball target exists and is tested.
7. A signed git tag on os384 points at the canonical commit, with release notes listing the hashes in §8.8.
8. Documentation names what is defended against and what is deferred, and a security-literate reader can locate each claim.

These are properties of the shipped artifact, not burn-down on a work plan.

---

11. Open questions (for team decision)

Listed here to be explicit about what still needs a call. These are decisions, not preferences.

- **O1: Release signing method.** cosign (keyless, OIDC, GitHub-native) vs. GPG (traditional, manual, portable). Affects operator workflow and the audit trail.
- **O2: Server-side setPage sanity verify.** Adds ~40 lines of worker code; catches malformed publishes early. Net benefit marginal for threat model; worth it for operator ergonomics?
- **O3: Default replay-enforcement behaviour.** Accept any contentVersion and expose it, or enforce monotonic pinning by default via localStorage or equivalent?
- **O4: Scope of pins.json.** Docker-only (services/docker/pins.json) or broader (services/pins.json) if other self-host paths adopt the pattern?
- **O5: Base-image digest pinning.** Pin FROM node:20@sha256:<digest> with a documented rotation cadence, or leave as floating node:20?
- **O6: Envelope byte-layout fine points.** (a) Signed region includes magic/version: yes/no? (b) Pubkey encoding: JWK JSON or raw EC point? (c) Explicit bodyHash field for streaming verification?
- **O7: Verifier-artifact URL policy.** Version-immutable (libLoader-v01.js, -v02.js, ...) or content-addressed (libLoader-<hash>.js)?
- **O8: Does RC3 ship a browser extension (Tier 3) or only Tiers 0–2 + 4?**
- **O9: Release-note publication channel.** GitHub Releases only, or also a dedicated "canonical hashes" page on 384.dev (chicken-and-egg, but could be Tier-1 readable)?
- **O10: Naming.** libLoader vs. signedPageLoader vs. something else? Envelope content-type: sb384signedV1 vs. alternatives?

Not on the list: **the existence** of the signed-envelope + separate-verifier approach. That is offered as Proposal A; if the team wants Proposal B, that needs its own section.

---

12. Preferences carried from the existing stack

Distinguished from §11 because these are not genuinely open — they follow directly from existing primitives and the platform's philosophy. They could be contested but would require justification:

- **P1: ECDSA P-384 with SHA-384** for signatures. Already the stack primitive.
- **P2: pageKey = base62(SHA-384(pubkey)) as enforced bootstrap binding.** Already the URL convention; promoting it to a security-bearing property is a labeling change.
- **P3: Breaking the on-the-wire page format is acceptable** given beta status and the magnitude of the gap being closed.
- **P4: Client-side verification is the load-bearing check;** server-side is at best defense-in-depth.
- **P5: Same envelope format and verifier across browser and Node runtimes.** Principle 5.
- **P6: Canonical artifacts are versioned and immutable once published.** Discipline for any SRI-pinned or hash-published artifact.

---

13. Deferred / out-of-scope (with rationale)

- **Key rotation and revocation.** Separate design, separate cycle. The mechanism will plausibly involve chained signing ("this new key is endorsed by the old key") or re-publication under a new pageKey with an announcement protocol; premature to commit now.
- **Strict replay enforcement.** Envelope carries the data; enforcement is a client-side policy layer. Designing the UX around "the library I have is older than the latest one claimed by the server" deserves its own attention, not a hasty default.
- **Transparency log [B§CT].** Zeroth-approximation replaced by signed git tags + release-note hashes; a real append-only externally-witnessed log is valuable but not on the critical path for closing the §6 gaps.
- **Reproducible builds.** Important; separate design.
- **Hardware-TEE attestation / remote attestation.** Not needed for the current threat model; revisit if/when os384 runs on TEE-capable devices.
- **Threshold / multi-sig.** Revisit once the single-sig story is wired end-to-end.
- **Post-quantum migration.** P-384 is not quantum-resistant. Industry-wide migration path is still being settled; when it converges, os384 follows.

Each deferred item is named so that a reader can ask about it without thinking it has been forgotten.

---

14. What this document is not

- Not a work plan. Effort and scheduling belong in separate artifacts.
- Not a threat model. Threat models enumerate adversaries, assets, and attacks; this document takes a more limited "classical problem → current gap → design principle" structure.

- Not a specification. Proposal A is concrete enough to review, not concrete enough to implement from. Field-level normative text is for the follow-on envelope spec and verifier API documents.
- Not a policy document. Issues like vulnerability disclosure, incident response, and coordinated rotation are named in §13 but not solved here.

Appendix A — os384 in context

For readers unfamiliar with the platform.

A.1 What os384 is, informally

os384 is a browser-native *sovereign computing* platform. The goal is to let individual users own their data, their identities, and the software that operates on their behalf, with as little reliance on third-party intermediaries as possible. It occupies a space adjacent to peer-to-peer networks, self-hosted software, and end-to-end encrypted messaging, but centred on the browser as the universal runtime.

Concretely, os384 is a small number of interlocking services and libraries, currently run on Cloudflare Workers for hosted convenience, that together provide identity, storage, publication, and interactive messaging — all cryptographically anchored to user-held keys.

A.2 Root identity: SB384

Every principal — a user, a service, an application — has an SB384 identity, centred on a P-384 ECDSA keypair. The public key has canonical derived forms: a base62-encoded short hash (pageKey, 21 characters, = base62(SHA-384(pubkey))) used in URLs and addressing; a longer hash form; and the raw JWK for verification operations.

Keys live under the user's control. The platform does not issue accounts; it recognises keypairs.

A.3 Shards

Shards are padded, content-addressed, AES-256-GCM encrypted binary blobs. They are the unit of opaque storage. A shard's address is determined by its ciphertext contents, so retrieval is naturally integrity-checked: given a shard address, you know whether the bytes you got back are the bytes that belong to that address. Shards are how data-at-rest is stored; the storage operator learns nothing about plaintext, and cannot substitute content without being detected.

A.4 Channels

Channels are owner-keyed, end-to-end encrypted, interactive endpoints backed by addressable workers on the channel server. They support real-time messaging, per-channel

storage, and a model of "everyone who has the right key can participate." The channel server brokers but does not read.

A.5 Pages

Pages are the open-web publication mechanism. A page is fetched at https://<host>/api/v2/page/<pageKey> where the pageKey is the short hash of the owner's public key. Pages are how os384 publishes things meant to be publicly readable: its own library code, its own service worker, application-facing content, landing pages. Today, pages are served as opaque bytes with a server-computed content hash used for caching; there is no verification of owner signature at fetch time. This is the primary gap this document addresses.

A.6 Services

The hosted stack runs several Cloudflare Workers:

- **channel server** (c3.384.dev etc.): brokers channels and pages.
- **storage server**: shard storage.
- **mirror server**: a local-first shard cache.
- **admin server**: local-dev-only utility for refreshing tokens.
- **loader** (deploy shell): serves the 384.dev entry HTML and related web assets.

A Docker compose stack in services/docker/ mirrors these for self-hosting — advanced users can run the whole thing on their own hardware.

A.7 Clients

- **Browser**: lib384, a TypeScript/ESM library loaded via the 384.dev entry page or bundled by applications directly.
- **CLI**: a 384 command-line tool for scripted publishing and administration, built on lib384.
- **Native**: integrations such as Cowork (a desktop tool) embed os384 primitives.

A.8 Philosophy, briefly

"Trust as little as possible." The platform is designed so that an honest user can reason about exactly who has to be trusted at each point, and — where technically feasible — replace ambient trust with cryptographic verification. The gap between the current implementation and this ambition is what this document is addressing.

Appendix B — Security concepts referenced in this document

For readers who are technically proficient but not security specialists.

B§SRI — Subresource Integrity

Subresource Integrity is a standard browser feature that lets HTML pin the exact byte-content of an external resource by hash. Syntax:

```
<script src="https://cdn.example.com/lib.js"
      integrity="sha384-<base64hash>"
      crossorigin="anonymous"></script>
```

The browser fetches the resource, computes its hash, and refuses to execute it if the hash does not match the integrity attribute. If a CDN is compromised and serves different bytes, the browser surfaces a loading error instead of running the attacker's code. SRI requires the crossorigin attribute for cross-origin fetches so that the browser can compute the hash.

SRI is a *hash pin*, not a signature check. It does not tell you who produced the bytes — only that the bytes match a hash you embedded in your HTML. The hash itself is the trust anchor.

Limitations: SRI does not cover the HTML itself (the HTML is what carries the SRI attribute; you cannot pin what you have not yet fetched). It does not cover dynamic imports unless the importer adds integrity. It does not help if the attacker controls the HTML source.

B§CSP — Content Security Policy

CSP is an HTTP header (or <meta> tag) that tells the browser what kinds of content a page is allowed to load and execute. A strict CSP can:

- Forbid all inline JavaScript, or permit only inline scripts with specific nonces or hashes.
- Restrict script sources to specific origins or hashes.
- Forbid loading fonts, images, frames from unlisted origins.
- Prevent the page from being framed by other origins.

CSP is enforced by the browser. A CSP delivered via response header is under the hosting provider's control; a CSP delivered via <meta http-equiv="Content-Security-Policy"> lives inside the HTML and travels with it.

Strict CSP is a meaningful defense against CDN-injected scripts: even if the CDN injects an inline <script> tag, the browser refuses to execute it if the CSP forbids inline scripts and the CDN cannot forge a valid nonce/hash.

CSP does not cover the HTML source itself, and a CDN that rewrites the CSP along with the rest of the HTML can weaken it. Delivering CSP in both header and <meta> tag gives two chances to catch tampering.

B§CT — Certificate Transparency (and analogues)

Certificate Transparency is a system where TLS certificate authorities append every issued certificate to public, append-only logs. Domain owners can monitor the logs for unexpected

certs issued against their domains; if a CA is coerced or compromised into issuing a rogue cert, the cert appears in the public log and the evidence is preserved.

The same pattern generalises: any security-bearing artifact (code signatures, release bundles, signed envelopes) can be logged to a publicly witnessed append-only log, giving the community a way to detect surprise issuance. **sigstore Rekor** is a notable implementation for code-signing artifacts.

When this document references "CT-like log," it means an append-only, externally-witnessed publication of canonical artifact hashes and signatures — a way for community auditors to notice if a platform operator quietly pushes out a different artifact than the one they announced.

B§SupplyChain — Supply-chain attacks

Even if a project's own source code is clean, its build can include code from dependencies, build tools, or the build environment itself. Attacks include:

- **Compromised maintainer account.** A dependency's maintainer is coerced, phished, or replaced (the xz-utils incident).
- **Typosquatting.** A malicious package with a name similar to a legitimate one gets installed by mistake.
- **Dependency confusion.** A private package name is registered publicly; build tools fetch the public (malicious) one instead.
- **Build-tool compromise.** A compromised compiler (XcodeGhost) or build-service injects code at compile time.
- **Post-install scripts.** Package managers that execute scripts during install can run arbitrary code in the developer's environment.

Defenses include lockfiles with integrity hashes, dependency auditing, signed packages, reproducible builds, and vendored dependencies.

B§ReproducibleBuilds — Reproducible builds

A build is "reproducible" if running it on any conforming environment from the same source produces bit-identical output. Reproducibility lets multiple parties verify that a published binary actually corresponds to a published source. Without it, there is a deniable gap between "source looks clean" and "published binary is clean."

Reproducible builds require eliminating sources of nondeterminism: timestamps, random IDs, filesystem ordering, locale, compiler version. Large ecosystems (Debian, Bitcoin Core) treat reproducibility as a first-order release property.

B§TOFU — Trust on First Use

TOFU is a trust model where the first time a client interacts with a service, it records the service's credential (public key, certificate, fingerprint) and from then on requires that credential to match. Common in SSH (known_hosts) and in some E2E messaging apps.

TOFU's weakness is the *first* interaction: if the first fetch is already compromised, the attacker's credential gets pinned.

Mitigations include out-of-band verification (comparing a fingerprint via a different channel), centralised attestation (certificates from a CA the client already trusts), and transparency logs.

B§CodeSigning — Code signing (overview)

"Code signing" names a family of mechanisms where a publisher cryptographically signs a piece of code, and a platform verifies the signature before running it. Prominent examples:

- **Apple Notarization / Gatekeeper.** macOS and iOS verify developer-ID signatures before allowing execution.
- **Windows Authenticode.** Executables signed with certs chained to Microsoft-trusted CAs get the "verified publisher" treatment.
- **Android APK signing.** APKs must be signed; Android verifies the signature during install.
- **GPG-signed packages.** Long-standing model for Linux distributions: packages signed with maintainer keys, keys signed with distribution root keys.
- **cosign / sigstore.** Modern keyless-signing approach where signing ties to an OIDC identity and publishes to a transparency log. Good fit for GitHub-based workflows.

All code-signing schemes bottom out in a trust root: a CA, a platform vendor, a key distributed with the OS, a certificate pinned to a domain. The root's integrity is ambient — outside the signing scheme itself. The question "who is authorised to sign what" is at least as important as the cryptography.

B§ECDSA — ECDSA over P-384

ECDSA (Elliptic Curve Digital Signature Algorithm) is a signature scheme. A P-384 ECDSA keypair consists of a private key and a public key on the NIST P-384 curve. The private key can produce signatures; anyone with the public key can verify them. os384 uses P-384 with SHA-384 hashing, which gives approximately 192 bits of classical security.

Practical properties:

- **Signatures are 96 bytes** raw (48-byte r + 48-byte s). Web Crypto exposes the raw form by default.
- **Verification requires the public key**, which is typically delivered in JWK format for web use.
- **Signatures do not prove recency.** A valid signature tells you the signer signed the bytes; it does not tell you when, so replay protection must be layered.
- **P-384 is not quantum-resistant.** Industry-wide PQ migration is an ongoing effort; nothing os384-specific about this exposure.

B§FirstVisit — Trust-on-First-Visit for web apps

A specialisation of TOFU. The first time a user visits a web app, the browser trusts DNS + TLS + the server, and loads whatever HTML comes back. Subsequent visits can use service workers, local caches, or browser extensions to harden the experience — but the first visit is always exposed to whoever controls that initial delivery path.

Defenses that help this case:

- **Bookmarking the HTML to disk** after verifying its canonical hash out-of-band.
- **Installing a browser extension** that bundles the entry HTML.
- **Installing a native app** that does the same at OS-codesigning integrity.
- **Community publication of canonical hashes** so that a careful user can verify what they fetched.

There is no web-platform primitive that solves first-visit integrity fully; the best available is "make the entry point small, auditable, and hash-published, and give paranoid users an escape hatch."

End of document.